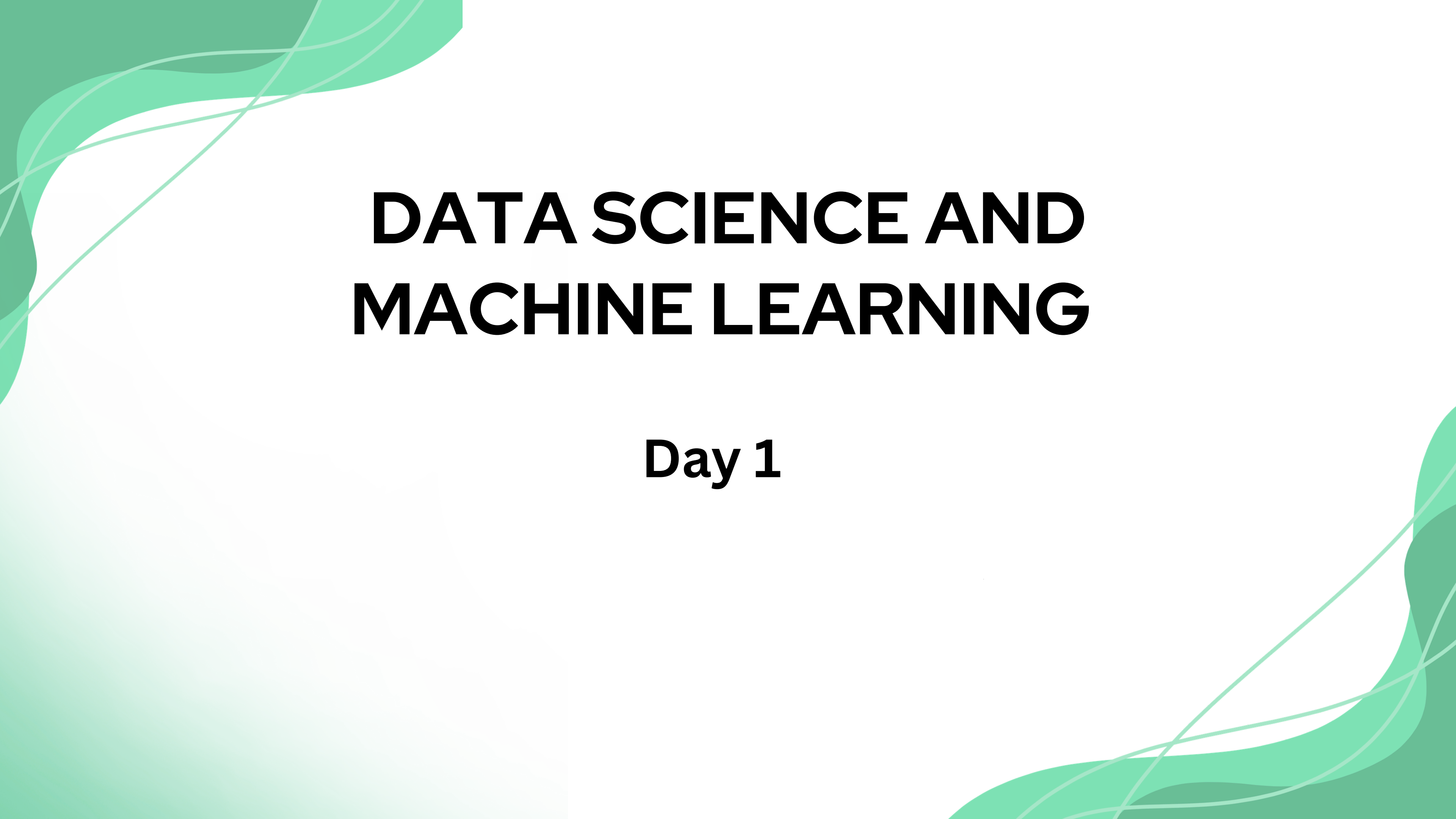




DATA SCIENCE AND MACHINE LEARNING

Day 1

Topics

Day 1

1. Installing Python

**2. Installing Python, IDE,
and JupyterLab.**

3 Install git

Learning Outcomes



- **Language (Python)**
- **Version Control (git)**
- **Terminal (linux or Bash Commands)**
- **Database**
- **Mathematics**

Programming Skills(Python)

- Data Structure
- loops
- functions
- OOP basics

Working with libraries

- NumPy(Arrays, math operations)
- Pandas(DataFrames, cleaning)
- Matplotlib/Seaborn(Line, bar, scatter/Beautiful visualizations)



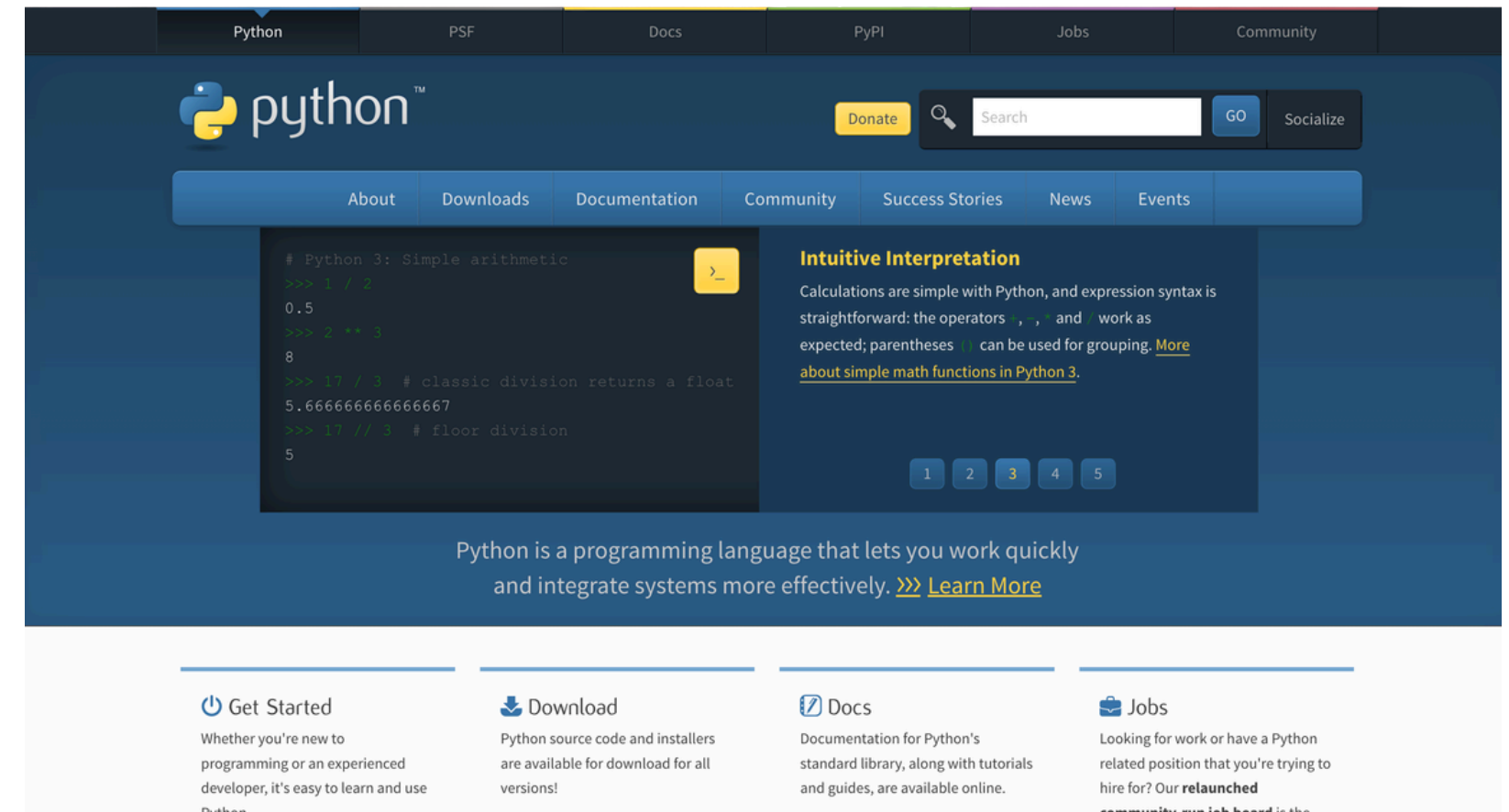
Introduction – Installing Python

Introduction – Installing Python

- Python
- Pip (comes with Python)
- IDLE or VS Code (optional editor)

Download Python

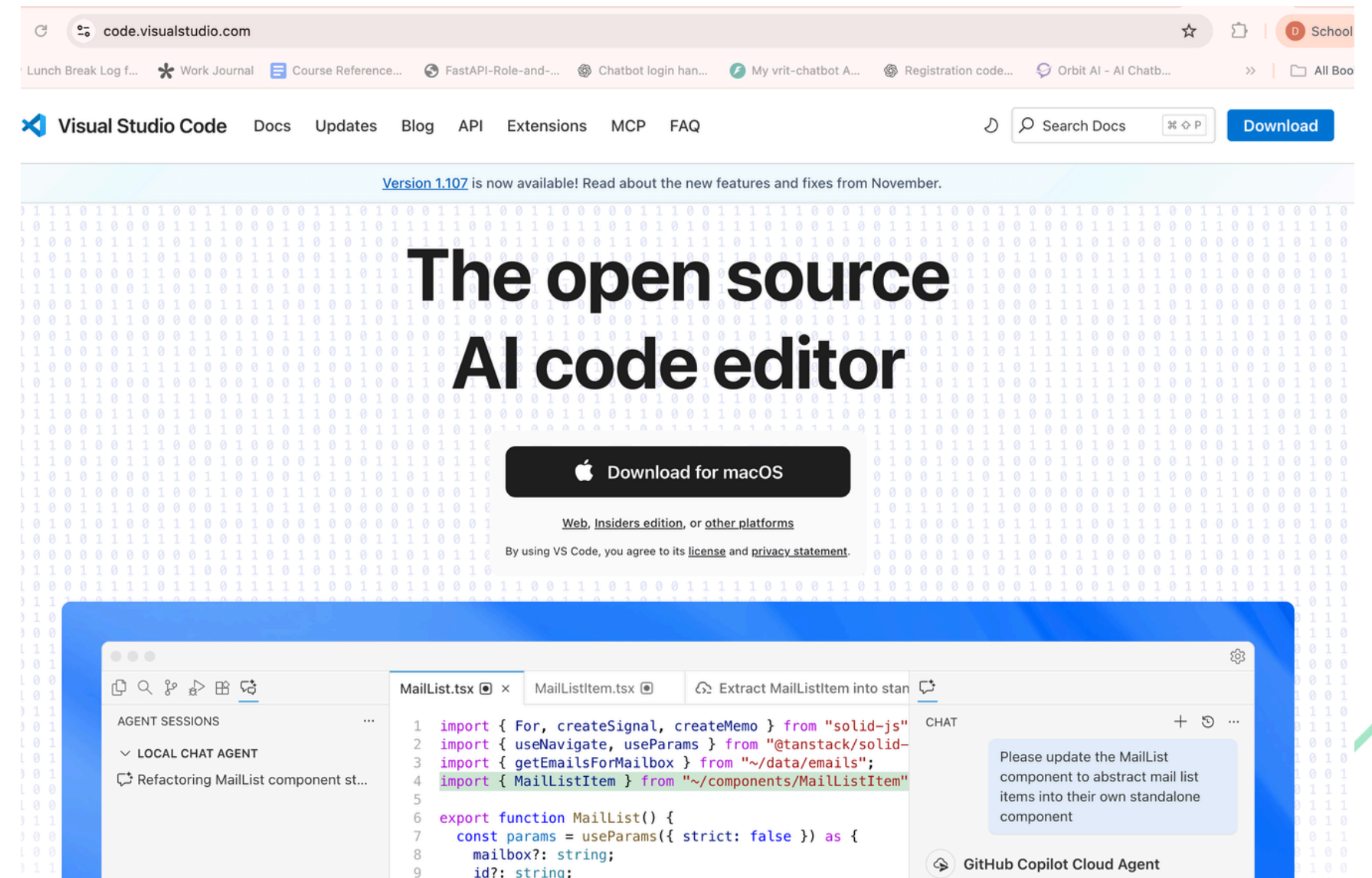
- Go to the official website:
- python.org → Downloads (<https://www.python.org/downloads/>)
- Click Download Python 3.x.x.
- “Add Python to PATH”



Introduction – Installing Vscode

Download visualstudio

- Go to the official website:
- <https://code.visualstudio.com/>
- Click Download



Verify Installation

Windows:

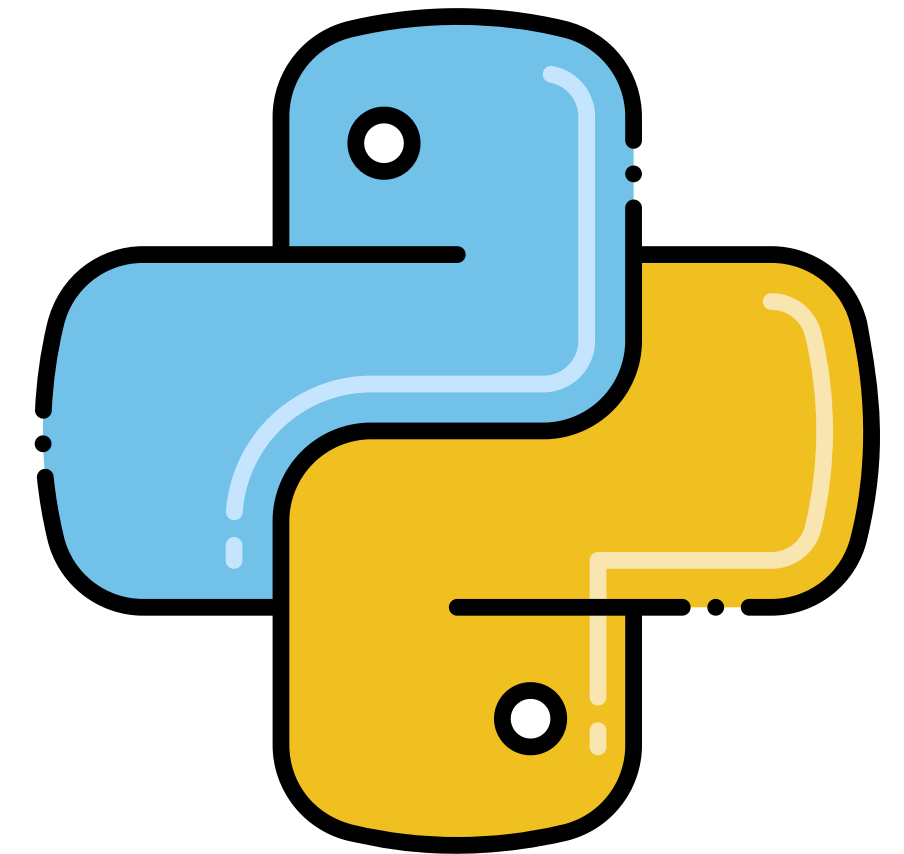
- Open Command Prompt
- Type:
- `python --version`
- `pip --version`

Mac/Linux:

- Open Terminal
- Type:
- `python3 --version`

First Program (Test)

- Open VS Code / IDLE
- Create a new file: `hello.py`
- `print("Hello, Python!")`
- Run `python hello.py`



Introduction – Jupyter Notebook & JupyterLab

Install Jupyter Notebook

`pip install notebook`

or

Install JupyterLab

`pip install jupyterlab`

Start Jupyter Notebook (<https://jupyter.org/install>)

(Recommended)

Run in Terminal

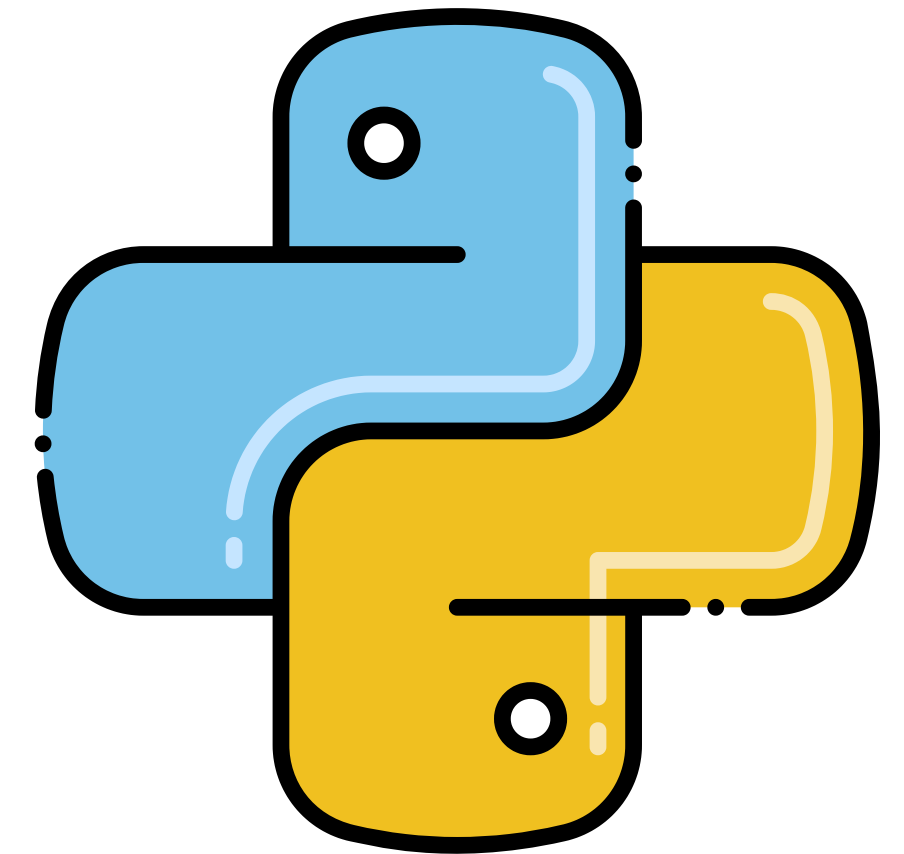
`jupyter notebook`

Start JupyterLab

`jupyter lab`

Using colab

<https://colab.research.google.com/>



Create a Virtual Environment

In Terminal / CMD, go to your project folder

```
python -m venv env
```

env = name of the environment (you can choose any name)

Activate the Environment

Windows

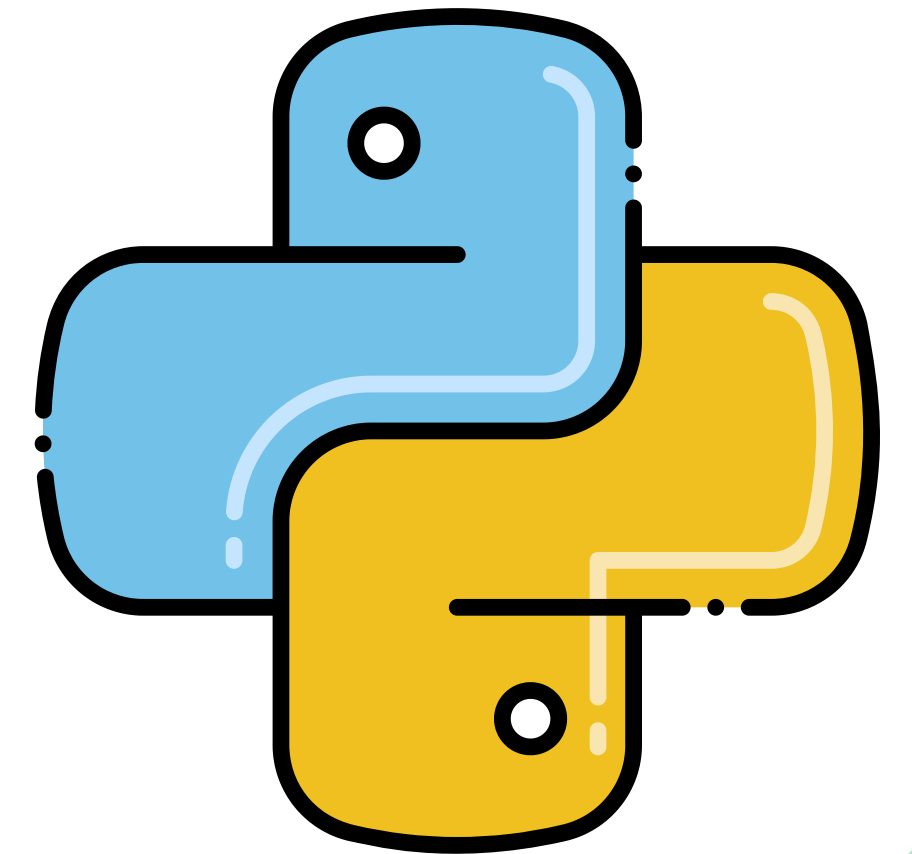
```
env\Scripts\activate
```

Mac / Linux

```
source env/bin/activate
```

Deactivate the Environment

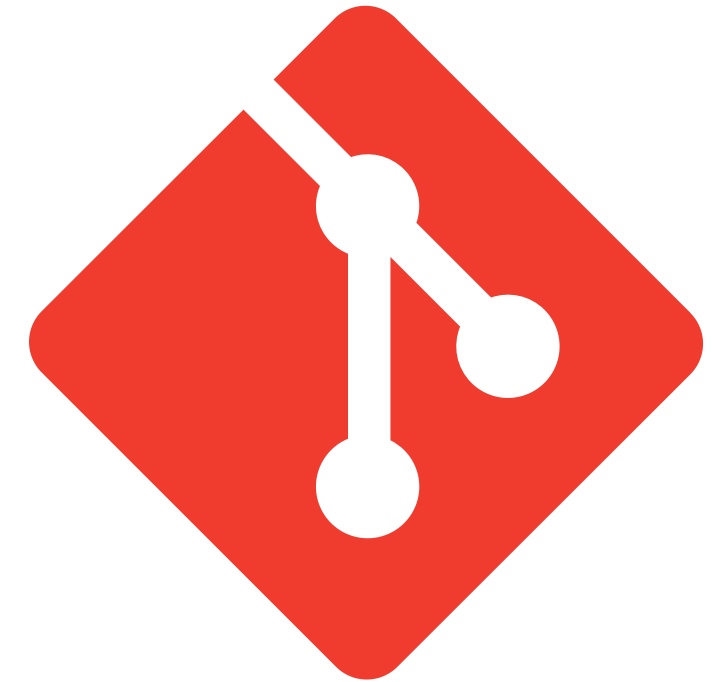
```
deactivate
```



Git/GitHub for version control

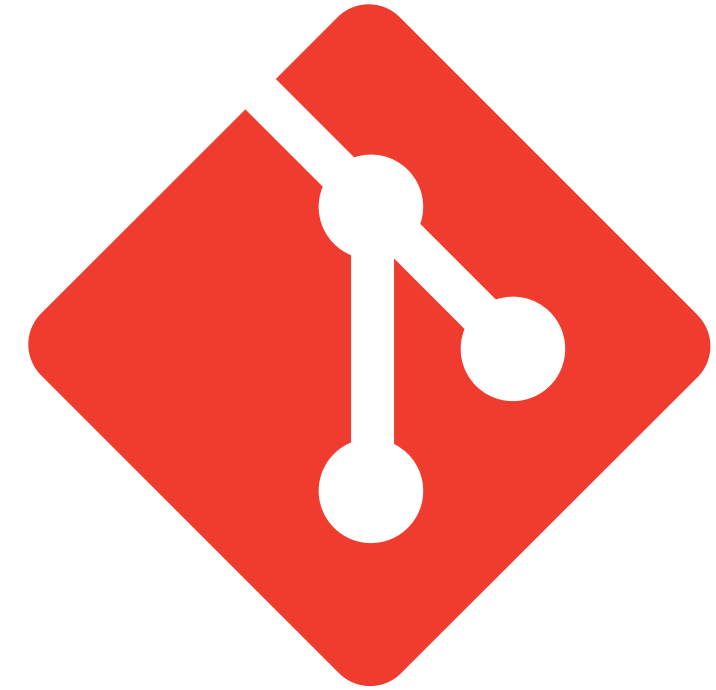
Git is a distributed version control system

- **Why Use Git?**
 - Keeps a history of all changes.
 - Enables collaboration without overwriting others' work.
 - Provides branching for experimenting safely.
 - Easy to undo mistakes and restore previous versions.
- **Core Concepts**
 - **Repository (Repo):** Project storage with full history.
 - **Commit:** A snapshot of changes.
 - **Branch:** A parallel line of development.
 - **Merge:** Combining changes from branches.
 - **Clone:** Downloading a copy of a repo.
 - **Push/Pull:** Uploading/downloading changes.



Git/GitHub for version control

- **git init** — Create a New Repository
 - Initializes Git in a folder.
 - Creates a hidden .git directory to start version control.
- **git clone** — Copy an Existing Repository
 - Downloads a remote repository (e.g., from GitHub) to your computer.
- **git add** — Stage Changes
 - Prepares modified files to be committed.
 - Moves changes to the staging area.
 - Example:
 - `git add file.txt`
 - `git add .` (adds all changes)
- **git commit** — Save a Snapshot
 - Records the staged changes permanently to the repository history.
 - `git commit -m "Initial commit"`
- **git push** — Upload Local Changes
 - Sends your local commits to a remote repository (GitHub, GitLab, etc.).
- **git pull** — Get Updates from Remote
 - Fetches new changes from the remote repository.
 - Automatically merges them into your local branch.



Git Playground

- <https://learngitbranching.js.org/>